

Unidad 5



Programación de equipos y sistemas industriales

Informática industrial





Índice

5.1. Software de programación

- 5.1.1. Lenguaje de bajo nivel o lenguaje máquina
- 5.1.2. Lenguaje ensamblador
- 5.1.3. Lenguaje de alto nivel
- 5.1.4. Proceso de programación

5.2. Programación estructurada

- 5.2.1. Algoritmos
- 5.2.2. Estructuras de control
- 5.2.3. Representación gráfica de los algoritmos

5.3. Pseudocódigo

- 5.3.1. Estructuras básicas

5.4. Lenguajes de alto nivel

- 5.4.1. Herramientas de desarrollo

5.5. Entidades que manejan los lenguajes de alto nivel

- 5.5.1. Constantes y variables
- 5.5.2. Estructuras de datos

5.6. Juego de instrucciones del lenguaje

- 5.6.1. Funciones
- 5.6.2. Sintaxis



Introducción

Como futuros técnicos en automatización, seguramente trabajaréis con máquinas o robots que necesiten de una programación, pero para eso, primero hay que saber que es programar. Dentro de la programación existe el lenguaje de programación en el que vamos a escribir las instrucciones y el código fuente que es el código resultante de escribir dichas instrucciones.

En esta unidad veremos los principales conceptos de programación, sus estructuras de datos fundamentales y en que algoritmos se suelen basar para trabajar, así como una pequeña representación gráfica.

Al finalizar esta unidad

- + Conoceremos los diferentes lenguajes de programación.
- + Podremos distinguir los diferentes tipos de datos y estructuras.
- + Seremos capaces de realizar diagramas de flujo de aplicaciones.
- + Seremos capaces de manejar el entorno de desarrollo.
- + Sabremos las reglas y las estructuras del pseudocódigo.
- + Sabremos desarrollar funciones de usuario y librerías.



5.1.

Software de programación

Un *software* de programación serán las herramientas como programas y aplicaciones que usarán los programadores para depurar, crear y mantener otros programas. Serán programas sencillos, como compiladores, depuradores, enlazadores, etc. que se combinarán entre sí para hacer una tarea.

Las instrucciones se formarán en dos partes:

- > **Código de operación:** será el encargado de mandar al microprocesador que tipo de operación tiene que hacer.
- > **Operando:** indicara los datos con los que realizaremos las operaciones, o las direcciones de donde sacar los datos.

Estas instrucciones a su vez podrán clasificarse según el tipo de operación que queremos realizar:

- > **Instrucciones lógicas:** hará las operaciones lógicas entre los operandos, como por ejemplo la suma lógica, etc.
- > **Instrucciones aritméticas:** hará las operaciones aritméticas entre los operandos, como por ejemplo la suma aritmética.
- > **Instrucciones de ruptura de secuencia:** harán referencia a las instrucciones de salto condicional e incondicional.
- > **Instrucciones de transferencia de datos:** ordenaran el paso inferior entre los distintos dispositivos.
- > **Introducciones de control:** controlaran el desarrollo del programa como finalizarlo, iniciarlos etc.

5.1.1. Lenguaje de bajo nivel o lenguaje máquina

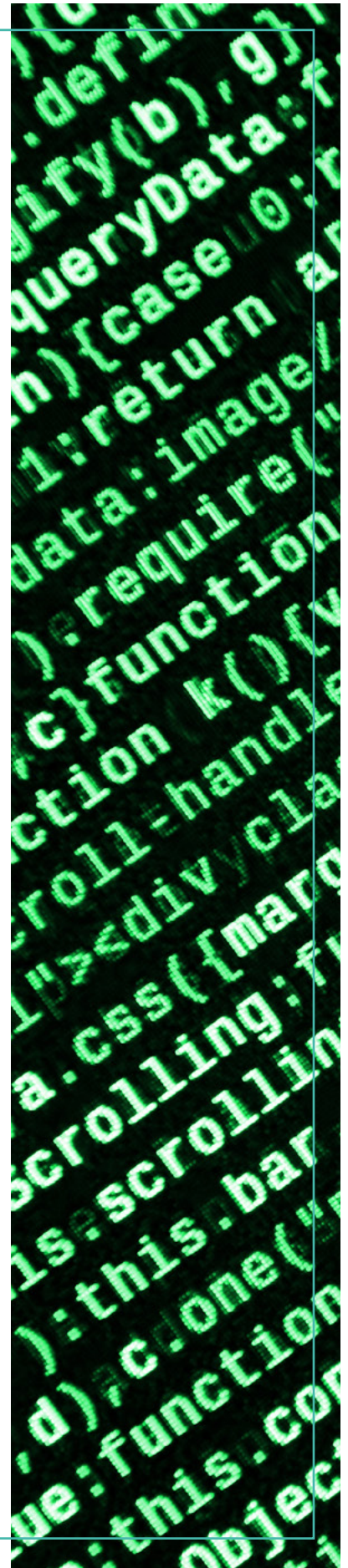
Será el lenguaje más difícil para entender, pero a su vez es el más cercano al sistema porque el microprocesador lo ejecuta directamente. Estará formado por palabras binarias y su longitud dependerá del microprocesador que usemos.

El problema de este tipo de lenguajes es que cada procesador usara su propio código máquina, así que no podremos intercambiar los sistemas microprogramables.

5.1.2. Lenguaje ensamblador

No se suele usar debido a que es un lenguaje de programación anticuado y tiene una elevada complejidad. Las instrucciones se representarán mediante nemónicos o combinaciones de letras.

Otro de los problemas que este tipo de lenguaje tiene es que es específico de cada arquitectura de computador física, mientras que los lenguajes de alto nivel serán portables.





5.1.3. Lenguaje de alto nivel

Se trata del lenguaje de programación más próximo al usuario ya que es el más evolucionado ya que no se basa en la arquitectura del sistema.

Las tareas serán el nombre de inglés de las acciones realizadas, pasando a depender del conjunto de órdenes del paquete software y no del repertorio de instrucciones del microprocesador, algunos ejemplos son: Cobol, Basic, C, Python.

Los lenguajes de alto nivel distan mucho de lo que entiende el sistema, así que para que el código se ejecute deben ser "traducidos" por lo que la ejecución es más lenta. El gran inconveniente de este tipo de lenguajes es que no se meten en lo más profundo del sistema, así que en algunas ocasiones añadiremos subrutinas realizadas en lenguaje máquina.

5.1.4. Proceso de programación

Una vez compuesto el programa (**código fuente**), habrá que transformarlo en código máquina (**código objeto**), que es lo único que comprende el sistema microprogramable. Para ello, se disponen de las siguientes herramientas:

- > **Ensambladores:** Su función es transformar el lenguaje ensamblador a código máquina.
- > **Compiladores e intérpretes:** Su función es transformar el resto de los lenguajes a código máquina.

La diferencia entre ellos consiste en que el compilador y el ensamblador leen varias veces el programa completo, producen el código máquina de todo el programa y lo ejecutan, por lo que la ejecución es más rápida.

El intérprete funciona de manera diferente, lee una línea, la convierte en código máquina y luego la ejecuta, después la segunda línea, y así con todo el programa.

Una vez transformado a programa objeto se encuentra limpio de errores. Si hay errores durante el proceso se tendría que editar el programa fuente, solventar el error indicado y repetir el proceso.

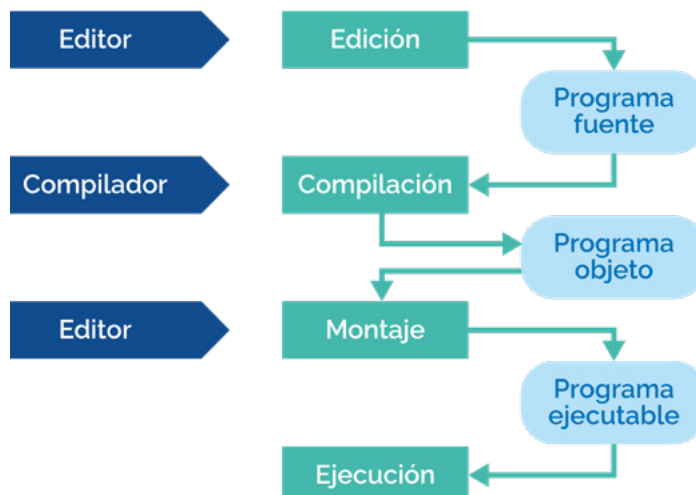


Imagen 1. Proceso de programación con un compilador

5.2.

Programación estructurada

Cuando los programas debido a la complejidad son cada vez más grandes, tendremos muchas más instrucciones. Los programadores pueden llegar a gestionar unos centenares de líneas de instrucciones, pero para resolver este problema se ideó lo que actualmente se conoce como programación estructurada y modular.

Al hablar de programación estructurada se hace referencia a un conjunto de técnicas:

- > Diseño descendente (top-down).
- > Posibilidad de descomponer una acción compleja en acciones más simples.
- > El uso de estructuras básicas de control (secuenciales, selectivas y repetitivas).

Si hablamos de programación modular nos referimos a la división de un programa en módulos (subprogramas), cada uno de ellos ejecutando una única tarea.

Cada módulo será programado y depurado con independencia del resto de los módulos.

Con la programación estructurada obtenemos las siguientes ventajas:

- > **Los programas son más fáciles de entender:** Un programa estructurado se puede seguir en línea, de arriba hacia abajo, sin necesidad de estar saltando de un sitio a otro.
- > **Se logra reducir el esfuerzo en las pruebas y en la actualización:** El seguimiento de los fallos o depuración (debugging) se convierte en un proceso más llevadero.
- > **Se crean programas más sencillos y rápidos.**

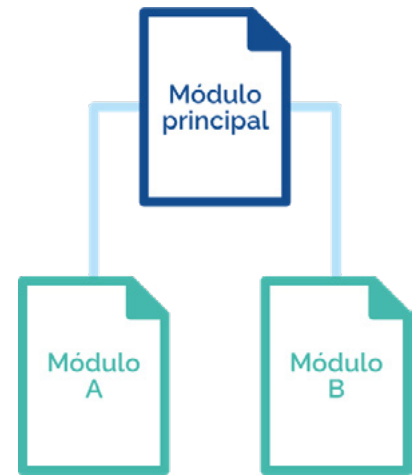


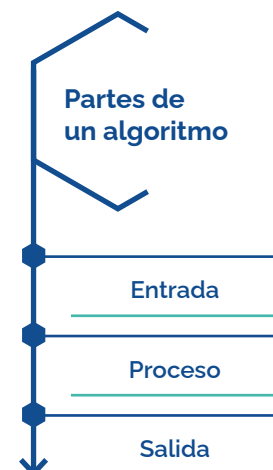
Imagen 2. Programación modular

5.2.1. Algoritmos

Un algoritmo es un conjunto de operaciones finitas, organizadas de manera lógica y ordenada permitiendo solucionar un determinado problema. Por lo que el diseño de todo algoritmo debe tener tres partes: entrada, proceso y salida.

Las características que debe seguir todo algoritmo son las siguientes:

- > **Conciso y detallado:** Debe reflejar con el máximo detalle el orden de ejecución de cada acción u operación.
- > **Flexible:** Que se le puedan realizar modificaciones o actualizaciones.
- > **Finito o limitado:** Un número determinado de pasos.
- > **Exacto y preciso:** El resultado debe ser el mismo ante los mismos datos de entrada.
- > **Claro y sencillo:** De fácil entendimiento y comprensión por parte del personal informático.



5.2.2. Estructuras de control

La programación estructurada hace uso de tres estructuras básicas de control:

- > **Estructura secuencial:** Las instrucciones de un programa se van sucediendo y ejecutando una después de la otra, en el mismo orden en el cual aparecen en el programa.
- > **Estructura selectiva:** Puede recibir el nombre de «si-verdadero-falso», da la opción de que existan dos alternativas en base al resultado de la evaluación de una condición (equivalente a la instrucción IF de todos los lenguajes de programación).
- > **Estructura repetitiva (o iterativa):** se le puede llamar «hacer-mientras-que», en ella se repite la misma instrucción mientras se cumpla una determinada condición.

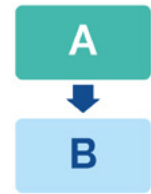


Imagen 3. Estructura secuencial



Imagen 4. Estructura selectiva



Imagen 5. Estructura repetitiva

5.2.3. Representación gráfica de los algoritmos

Para realizar algoritmos se usan técnicas de representación gráfica, una de esas técnicas son los denominados diagramas de flujo que usan símbolos estandarizados unidos por flechas que muestran la secuencia lógica de las operaciones y acciones que se deben realizar.

Los diagramas de flujo pueden estructurarse en dos grandes grupos: organigramas y ordinogramas.

Organigramas

Los organigramas o diagramas de flujo de sistemas o diagrama de flujo de configuración son representaciones gráficas del flujo de datos e informaciones entre los periféricos o soportes físicos que utiliza un programa.

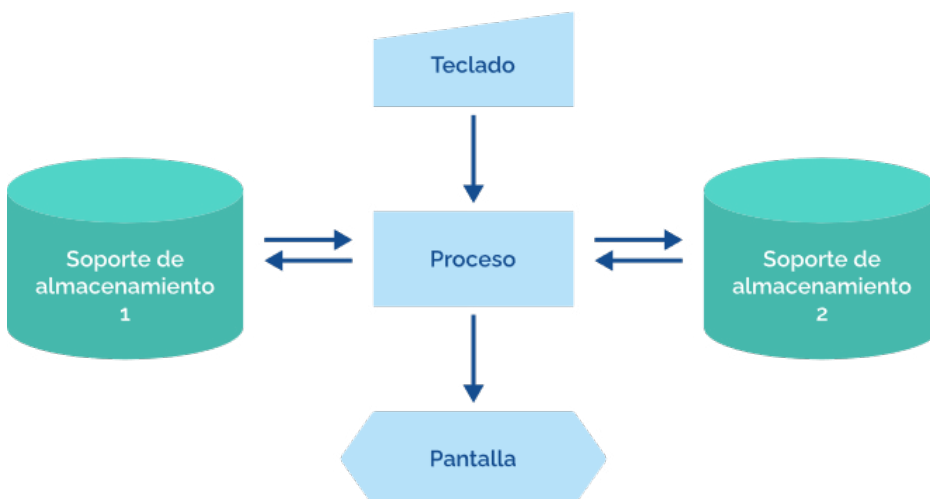


Imagen 6. Ejemplo de un organigrama



Ordinogramas

Los ordinogramas o diagramas de flujo de programas son representaciones gráficas que muestran la secuencia detallada de las operaciones que se van a ejecutar en el programa.

El diseño de todo ordinograma tiene un comienzo de la ejecución del programa que tendrá la palabra Inicio. Se detallará la secuencia de operaciones, siguiendo siempre en el orden que se deberán ejecutar (de arriba-abajo y de izquierda-derecha) siendo la palabra Fin que marca la finalización de ejecución del programa.

Existen unas normas básicas que hay que seguir:

- > Los símbolos se unen por medio de líneas de conexión.
- > Las líneas no se pueden cruzar.
- > A un símbolo pueden llegarle varias líneas de conexión, pero saldrá una del símbolo, salvo que este sea de decisión, que entonces saldrán tantas líneas como posibilidades existan.
- > Al símbolo de inicio no llega ninguna línea de conexión y de él únicamente puede salir una.
- > Al símbolo de final de proceso pueden llegar muchas conexiones, pero de él no puede partir ninguna.

Símbolo	Función
	Marca el inicio, el final o una parada necesaria (abre y cierra el diagrama).
	Representa un proceso u operación en general.
	Representa un subprograma o subrutina. Es decir, un módulo independiente que realiza una tarea.
	Decisión: Indica operaciones lógicas o comparativas, seleccionando en función del resultado entre varios caminos.
	Conector: Este símbolo es utilizado para el reagrupamiento de las líneas de flujo.
	Operación de entrada/salida en general: Sirve para la introducción de datos desde un periférico o la salida de datos a un periférico.
	Representan las flechas indicadoras de la dirección del flujo de los datos.



5.3.

Pseudocódigo

Se define como el lenguaje intermedio entre el lenguaje de las personas y el lenguaje de programación en el que vayamos a trabajar.



EJEMPLO DE PSEUDOCÓDIGO:

Proceso Resta

Escribir "Ingrese el primer número:"

Leer A

//Pide al usuario que ingrese un número, lee el número ingresado y lo almacena en la variable A

Escribir "Ingrese el segundo número:"

Leer B

//Pide al usuario que ingrese un número, lee el número ingresado y lo almacena en la variable B

$C \leftarrow A - B$

Escribir "El resultado es: ", C

//Resta las variables A y B y muestra el resultado al usuario

FinProceso

5.3.1. Estructuras básicas

En la redacción de cualquier pseudocódigo se usan tres tipos de estructuras básicas de control: secuenciales, selectivas e iterativas.

Estructuras secuenciales

Las instrucciones van unas tras otras en una secuencia fija que viene dada por el número de línea.

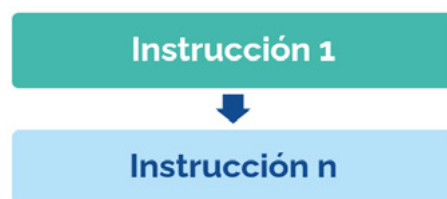


Imagen 7. Estructura secuencial

Estructuras selectivas

Son aquellas instrucciones que controlan la ejecución o la no ejecución de una o más instrucciones en función de que se cumpla o no una condición previamente establecida.

- > **Selectiva simple:** Las instrucciones selectivas son aquellas que pueden llegar a ejecutarse o no, según la condición.

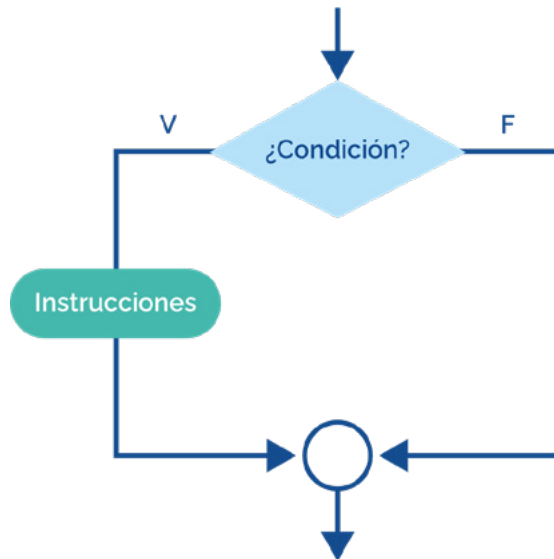


Imagen 8. Selectiva simple

- > **Selectiva doble (alternativa):** Realiza una instrucción de las posibles, según la condición. La condición es una variable booleana o una función reducible a booleana (lógica, verdadero/falso).

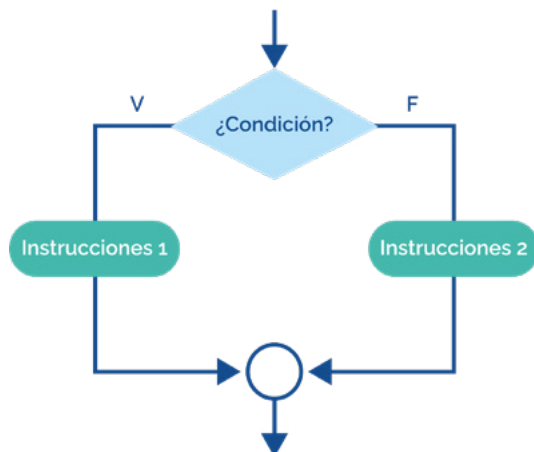


Imagen 9. Selectiva doble

Estructuras iterativas

Las instrucciones iterativas alteran la secuencia normal de ejecución del programa, permitiendo que un grupo de instrucciones se ejecute más de una vez de forma consecutiva, pueden recibir el nombre de bucles o lazos.

Bucle mientras: El bucle se repite cuando la condición sea cierta, sino no, al llegar al bucle si la condición es falsa, el bloque de instrucciones no se ejecutará.

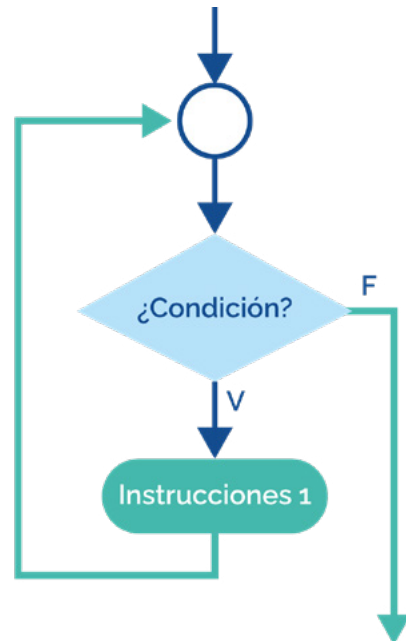


Imagen 10. Iterativa del tipo bucle

Bucle repetir-hasta que: Se utiliza cuando es necesario que el bloque de instrucciones se ejecute, al menos, una vez y hasta que se cumpla la condición.

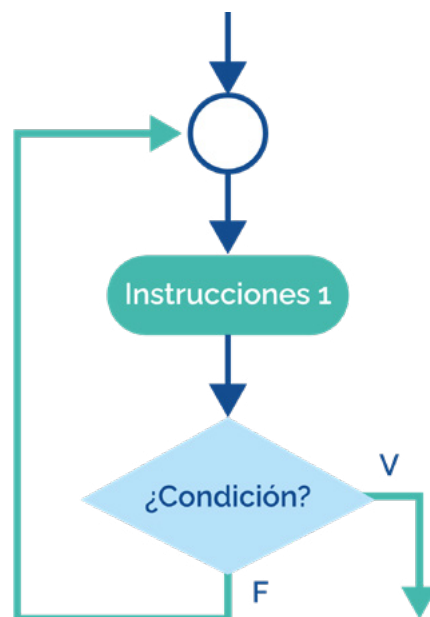


Imagen 11. Iterativa del tipo bucle repetir-hasta que



5.4.

Lenguajes de alto nivel

Cuando hablamos de lenguaje de alto nivel nos referimos al nivel más alto de abstracción de programación.

Pasaremos de tratar con registros, direcciones de memoria y pilas de llamadas a las variables, matrices, objetos, funciones, bucles, etc., priorizando la facilidad de uso sobre la eficiencia óptima del programa.

Con esto conseguiremos independizar el programa del soporte que lo ejecuta, así se podrá usar en cualquier equipo.

El único requisito será tener un programa traductor o compilador para conseguir el programa ejecutable en lenguaje binario de la máquina que se trate.

Las ventajas de este tipo de programación de alto nivel son las siguientes:

- > Crea un código más sencillo y comprensible.
- > Redacta un código válido para diversas máquinas o sistemas operativos.
- > Da la opción de crear programas complejos con menos líneas de código.

5.4.1. Herramientas de desarrollo

Los programas o aplicaciones necesarios para la implementación de un programa se pueden catalogar como:

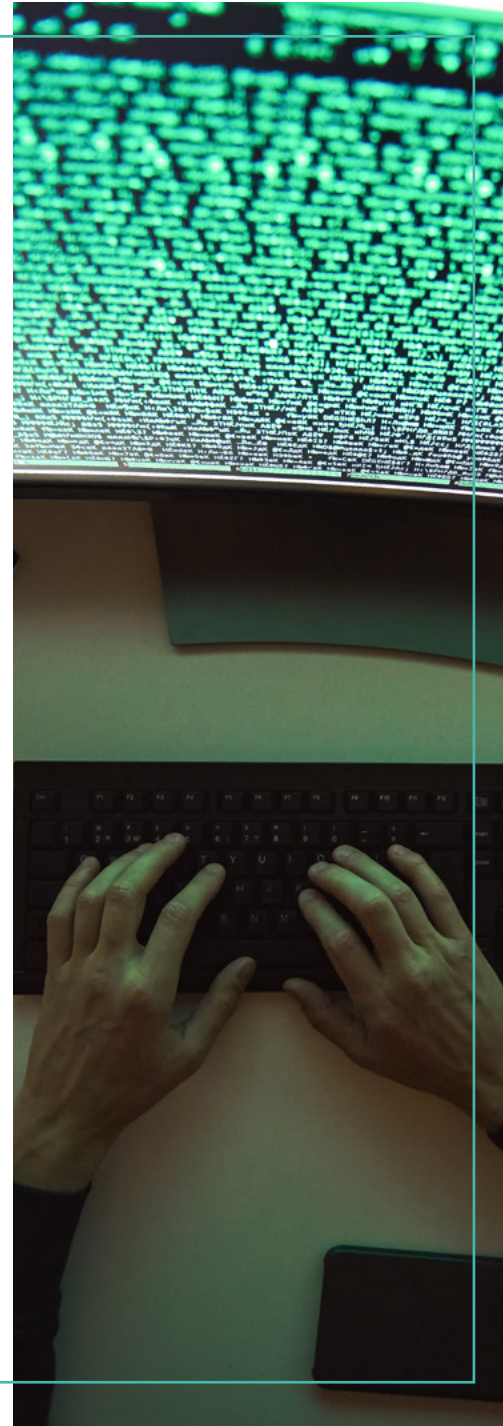
Editor de texto

En él escribimos el programa y generamos el código fuente, eligiendo el lenguaje de programación que queramos. Uno de los más usados es Notepad++.

Compilador

Con el compilador traducimos el fichero de código fuente de cualquier lenguaje de programación a su homólogo en código máquina.

Los compiladores más importantes son: GCC (para C), G++ (para C++), etcétera.





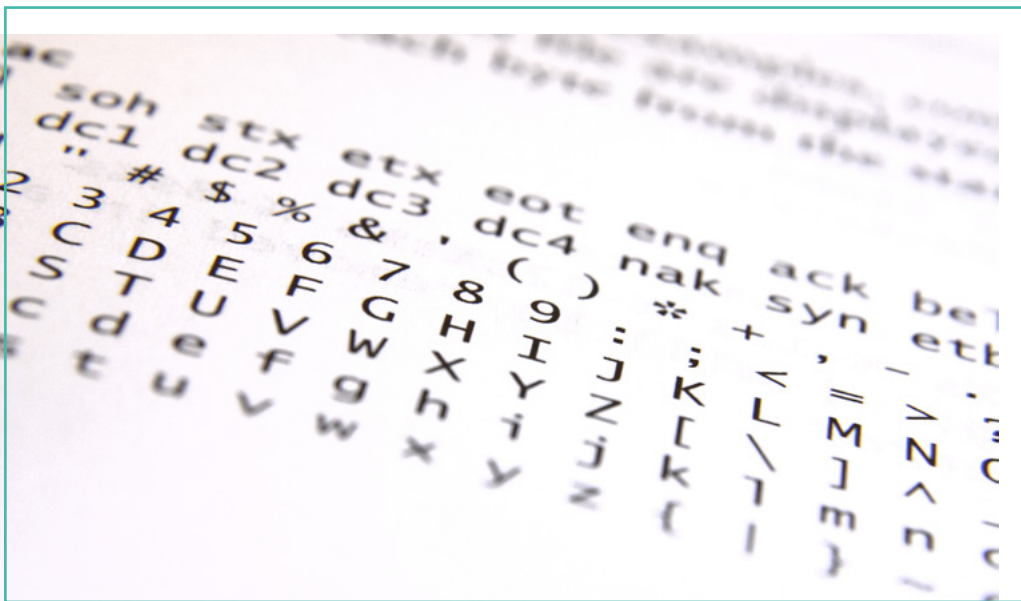
5.5.

Entidades que manejan los lenguajes de alto nivel

Para el sistema informático todos los datos son numéricos (series de unos y ceros). Por tanto, los datos tendrán que estar organizados de tal forma que los programas sean capaces de realizar los tratamientos necesarios de una forma determinada. Los datos llevan asociados un identificador, un tipo y un valor.

- > **Identificador:** Nombre para identificar el dato. El nombre puede estar formado por letras y números, es recomendable que el nombre asignado debe tener relación con la información que contiene.
- > **Tipo:** Rango de valores que podría tomar el dato. El tipo determina el espacio reservado para el dato.
- > **Valor:** Contenido que hay en el dato y tiene que estar dentro del rango de valores del tipo definido. En función del tipo, los datos se pueden clasificar en:
 - » **Datos básicos:** Son los numéricos, caracteres y lógicos. Los numéricos se usan para guardar magnitudes.

Los caracteres se emplean para representar un carácter en función de una tabla de representación (por ejemplo, código ASCII).
 - » **Datos derivados (punteros):** Guardan la dirección de memoria de otra variable.
 - » **Datos estructurados:** Son estructuras diseñadas para la agrupación de varios datos básicos o derivados. Por ejemplo, una tabla tridimensional de números imaginarios.



5.5.1. Constantes y variables

Las constantes son datos cuya información será fija mientras se realice la ejecución del programa. En el caso de las variables son datos que van siendo modificados durante la ejecución del programa.

5.5.2. Estructuras de datos

Las estructuras de datos nos ayudan a gestionar la información mediante el uso de tablas (arrays en inglés), que nos sirven para tratar gran cantidad de datos de forma ordenada, como las bases de datos.

Tablas unidimensionales

Normalmente conocidas como vectores, son conjunto de datos donde todos los elementos son del mismo tipo y características. Se le otorga un nombre identificativo común a los valores que guarda.

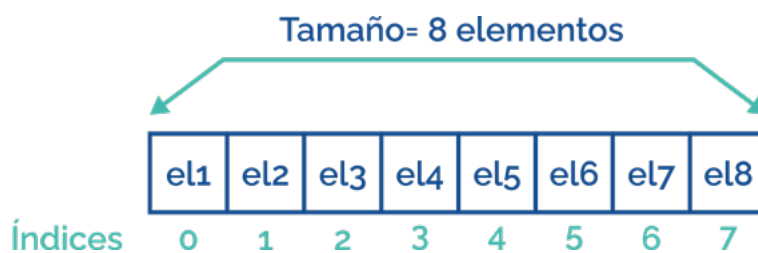


Imagen 12. Vector

Tablas bidimensionales

Este tipo de estructuras también reciben el nombre de matrices, son conjunto de datos donde todos los elementos son del mismo tipo y características.

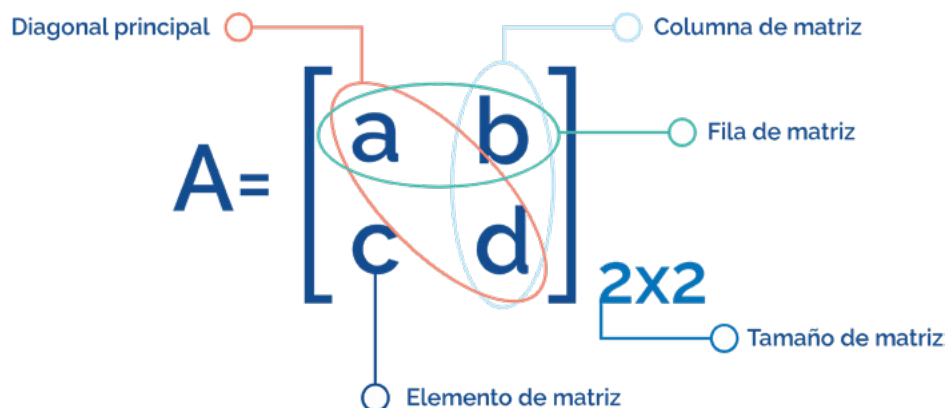


Imagen 13. Matriz

5.6.

Juego de instrucciones del lenguaje

Un juego de instrucciones del lenguaje indica las instrucciones con las que un lenguaje de programación funciona. Los juegos de instrucciones tendrán estas características.

5.6.1. Funciones

Una función es un conjunto de instrucciones que forman una unidad de código y su objetivo es evitar tener que repetir instrucciones constantemente.

De este modo, cuando invocamos a una función, lo que se realiza es ejecutar un determinado código predefinido.

Existen varias clasificaciones, pero a vamos a tomar en cuenta si la función puede retornar un valor o no:

- > **Funciones que no retornan un valor (procedimientos):** La función es vacío (void) y ejecuta una tarea con o sin paso de parámetros, pero no retorna ningún dato.
- > **Funciones que retornan un valor:** El tipo de la función es el mismo que el valor que hay que devolver.

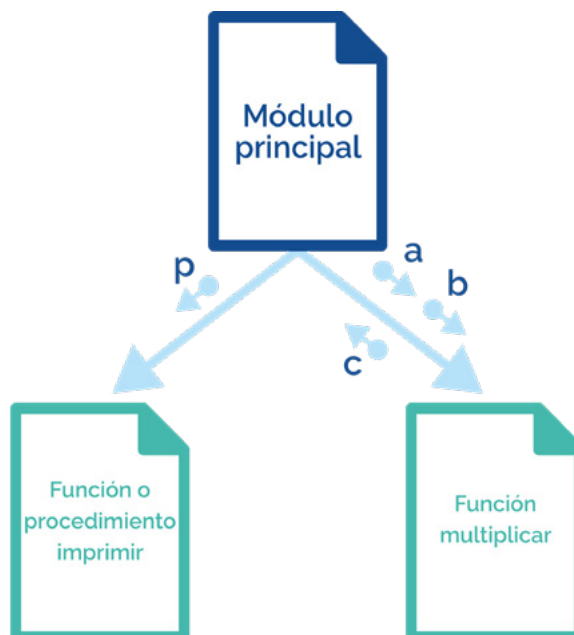


Imagen 15. Llamada de funciones con parámetros

5.6.2. Sintaxis

La sintaxis es la parte visible de un lenguaje de programación. Muchos de los lenguajes de programación son puramente textuales, es decir, usan líneas de texto que incluyen palabras, números y puntuación, aunque existen otros que no funcionan por líneas de texto sino por bloques.

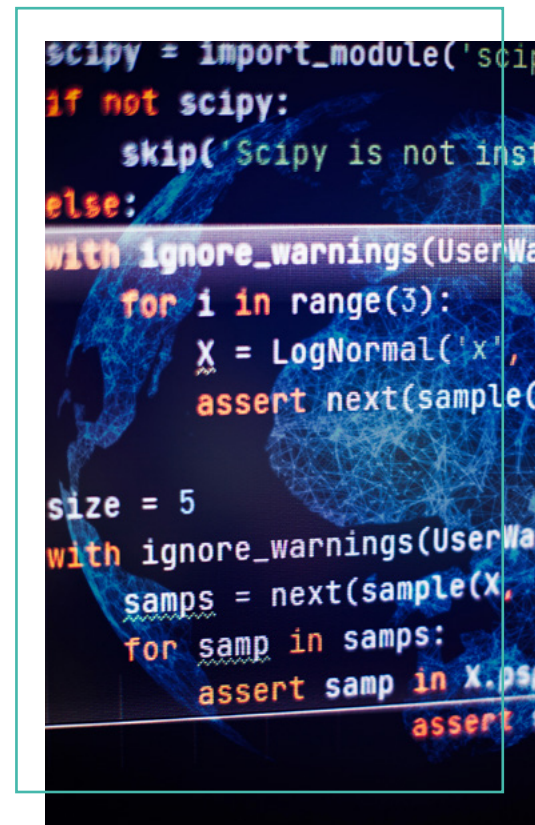


Imagen 14. Imagen 1. Sintaxis



 www.universae.com

